



FACULTY OF SCIENCES OF THE
UNIVERSITY OF PORTO

MASTER'S DEGREE IN INFORMATION SECURITY

Systems & Data Security

Public Ledger for Auctions over Kademlia

Marco Carneiro (up201908329)

Rui Fernandes (up202103071)

May, 2023

Abstract

In this report, we briefly describe the application we developed and discuss the decisions we made.

Contents

1	Introduction	1
2	Problem & Proposed Solution	1
3	Kademlia Network	1
3.1	Sybil & Eclipse Attacks	2
4	Public Ledger	2
5	Reputation System	2
6	Auction System	3
7	Conclusion & Future Work	3

List of Figures

Acronyms

DPoS	Delegated Proof-of-Stake
P2P	Peer-To-Peer
PoS	Proof-of-Stake
RPC	Remote Procedure Call
UTXO	Unspent Transaction Output

1 Introduction

Kademlia [1]

This report is structured as follows: §3 describes the implementation of a Kademlia P2P network and some optimizations that were implemented. §5 discusses some challenges that come with the potential implementation of a reputation system. Finally, §7 concludes the report.

2 Problem & Proposed Solution

Our implementation is divided in three parts:

- **P2P network:** The Peer-to-Peer network should allow a node to join it and gossip other nodes using the Kademlia algorithm.
- **Public Ledger:** The public ledger should be able to store auction transactions.
- **Auction Service:** The auction services gives the ability to a node to broadcast an asset it wishes to sell. Other nodes use this service to bid on the assets being sold.

3 Kademlia Network

In addition to the four RPC of the Kademlia protocol, we implemented two additional RPCs: one to broadcast a message to all the nodes in the network, and one to send a message to a specific node. The Kademlia implementation follows its specification which dictates the usage of four main processes. These would allow the network to gossip the existence, status and distance of nodes within it.

The main four RPCs are: PING, STORE, FIND_NODE and FIND_VALUE. In order, these allow to check for the status of a node, store given information on nearest nodes, find a specific node on the network by replying which nodes are closest to the requested key, and to find a specific value in the storage of the recipient of the request. These RPCs calls were implemented using gRPC for remote communication and its underlying Netty abstracted implementation.

- Concurrency parameter α

In addition, we also implement *optimized contact accounting*. In other words, updating the k -buckets frequently can result in a lot of traffic, which can lead to network congestion and slow performance. *explicar melhor*. This allows for temporary network disruptions and delays without permanently removing a node from the routing table.

3.1 Sybil & Eclipse Attacks

S-Kademlia [2].

To prevent Sybil attacks [3], When a new node joins the network, their node ID must be mined in a process that involves **Completar**. This ensures that it is not computationally feasible for an attacker to flood the network with nodes under their control, as the process of joining the network requires a significant amount of work.

4 Public Ledger

Every transaction is added to a transaction pool containing pending transactions, which will then be used to generate a new block and add it to the ledger.

As such, the balance of a wallet is the sum of all **UTXOs** relative to that wallet. By following this approach, previously spent transaction outputs are not accepted in subsequent transactions, which solves the *double spending problem*.

Due to timing constraints, we were not able to implement **DPoS**. Instead, we implemented **PoS**. In this case, we consider that all nodes in the network can act as validators. Then, when validating the transactions of a block, we choose a validator “at random”, taking into account their stake in the network – i.e. nodes with a higher stake are more likely to be chosen as validators. Nevertheless, to implement Delegated Proof-of-Stake (**DPoS**), it would suffice to **completar**.

5 Reputation System

Due to time limitations, we were not able to implement a reputation system. Regardless, our idea was to implement a system that allowed us to identify unreliable nodes within the network. As a consequence, these nodes would be used less often for routing and would potentially be removed (**ignorados pq nao da para remover um nodo ?**) altogether from the network.

To achieve this, we would compute and publish a **reputation** score to each node in the network, which would be then updated based on the behaviour of the corresponding node – deviating from the protocol would lead to a decrease in reputation, whereas adherence to the protocol would result in an increase in reputation. While implementing such a system may seem a simple task, it is by no means a trivial task due to several intricacies.

For example, there is the possibility of a node accumulating excessive positive reputation, which would potentially allow it to perform negative actions without a significant penalty. To address this issue, different approaches can be considered. We could, for example, implement a system where it becomes increasingly difficult to gain reputation for nodes with high and low reputation, which would make it harder to accumulate reputation and more challenging to

recover from negative actions. On the other hand, we could also cap the maximum reputation value to prevent this issue. Finally, we could implement a system where the penalty for malicious actions increases with a higher reputation score. Ideally, a combination of these approaches should be considered.

In addition, such a reputation system should balance accuracy and efficiency. In other words, it should be able to accurately capture a node's behavior while maintaining low overheads in terms of computation, communication and storage. Finally, it should also be designed to prevent malicious nodes from cooperating to improve each other's reputation or harm the reputation of other nodes.

6 Auction System

Originally, a solution similar to Peermart [4] was considered due to its algorithmic agnostic properties. A simpler solution was instead followed that follows the English Auction model just the same, with nodes bidding on a specific asset advertised by another node. The node which initiates the auction process by advertising an item to sell, is responsible from determining the best buyer and, upon completion of the auction, requesting the agreed payment. **complete**

7 Conclusion & Future Work

Unfortunately, due to **timing constraints/time limitations**, we were not able to test our application using Google Cloud's virtual machines. Nevertheless,

... using an identity-based approach, such as the one discussed in [5].

References

- [1] Petar Maymounkov and David Mazieres. Kademlia: A peer-to-peer information system based on the XOR metric. In *Peer-to-Peer Systems: First International Workshop, IPTPS 2002 Cambridge, MA, USA, March 7–8, 2002 Revised Papers*, pages 53–65. Springer, 2002.
- [2] Riccardo Pecori. S-Kademlia: A trust and reputation method to mitigate a Sybil attack in Kademlia. *Computer Networks*, 94:205–218, 2016.
- [3] John R Douceur. The sybil attack. In *Peer-to-Peer Systems: First International Workshop, IPTPS 2002 Cambridge, MA, USA, March 7–8, 2002 Revised Papers 1*, pages 251–260. Springer, 2002.
- [4] David Hausheer and Burkhard Stiller. Peermart: The technology for a distributed auction-based market for peer-to-peer services. In *IEEE International Conference on Communications, 2005. ICC 2005. 2005*, volume 3, pages 1583–1587. IEEE, 2005.
- [5] Luca Maria Aiello, Marco Milanese, Giancarlo Ruffo, and Rossano Schifanella. Tempering Kademlia with a Robust Identity Based System. In *2008 Eighth International Conference on Peer-to-Peer Computing*, pages 30–39, 2008.